

ICEPOT — AI Salesforce User Onboarding & Access Request Assistant

Instruction

Help me design the architecture for an **AI-powered Salesforce User Onboarding & Access Request Assistant** based strictly on the business processes, input sources, and outputs described below.

You are acting as the **Chief AI Architect** with 10+ years of experience designing enterprise AI applications, GenAI solutions, RAG systems, AI agents, workflow automation, and enterprise support applications.

This is a **home assignment from my GenAI course at TestLeaf** that I need to adapt to my **current Huron project scenario**.

The objective is to design a realistic AI solution that can assist the Salesforce support team in analyzing and processing Salesforce user onboarding and access-request tickets received through **Zoho Service Desk**.

1. IMPORTANT SCOPE

The application is **NOT a Test Case Generator**.

Do NOT design the solution primarily around:

- Test-case generation
- Automated regression testing
- Selenium/Playwright automation
- Test automation frameworks
- QA test management

The primary objective is:

Use Generative AI/RAG/Agentic capabilities to assist Salesforce support engineers in understanding user-request tickets, identifying missing information and approvals, determining the next required action, and preparing appropriate requester/manager communications.

The application should support the existing human-driven Salesforce support process.

2. Current Project Background

I work in a Salesforce support environment at Huron.

The team uses **Zoho Service Desk** for ticket management.

One of the important support activities is **Salesforce user creation and access management**.

The support team receives different types of tickets related to Salesforce users, including:

- Onboarding tickets
- Salesforce New User Request templates

- Salesforce user requests raised as incidents

The support team needs to review the ticket, validate required information, check approvals and conditions, communicate with requesters/managers when information is missing, and finally create the Salesforce user when all required conditions are satisfied.

3. Business Workflow A — Onboarding Ticket

An onboarding ticket is received in Zoho Service Desk.

The current process is:

Onboarding Ticket



Check Main Ticket



Check Salesforce Task



Review Required Information



Check Mirror User



Check Business Justification



Check Chatter User / Status



Manager Approval if Applicable



All Required Information Available?



Salesforce User Creation

Detailed process:

1. Receive onboarding ticket.
2. Open the main ticket.
3. Check whether a Salesforce task exists in the ticket.
4. Review the Salesforce task.

5. Check whether a mirror user has been provided.
6. If the mirror user is missing:
 - Request the mirror user from the requester.
7. Check whether business justification has been provided.
8. If business justification is missing:
 - Request business justification from the requester.
9. Once business justification is received, check whether the requested user is a Chatter user.
10. If the user is a Chatter user and the status is Pending:
 - Send/request the required approval from the user's manager within the same ticket.
11. Once all applicable information and approvals are available:
 - Proceed with Salesforce user creation.

The AI assistant should help the support engineer understand the current state and determine the appropriate **next action**.

4. Business Workflow B — Salesforce New User Request

Another request is received through the Salesforce New User Request template.

The process is:

Salesforce New User Request



Check Stage 1 Approval



Check Stage 2 Approval



Check Mirror User



Check Chatter Condition



Check Required Approval



All Conditions Satisfied?



Salesforce User Creation

Detailed process:

1. Receive Salesforce New User Request.
2. Check Stage 1 approval.
3. Check Stage 2 approval.
4. User creation should proceed only when the required approval stages are completed.
5. Check whether a mirror user is provided.
6. If mirror user is missing:
 - Request mirror-user information.
7. Check whether the requested user is a Chatter user.
8. If the user is a Chatter user and the status is Pending:
 - Request/validate the required Chatter-user approval from the appropriate manager.
9. If the user is not a Chatter user:
 - The Chatter approval condition does not apply.
10. Once all applicable information and approvals are completed:
 - Proceed with Salesforce user creation.

The AI assistant should identify:

- Current request state
- Missing information
- Missing approvals
- Applicable conditions
- Next required action

5. Business Workflow C — Salesforce Request Raised as Incident

Sometimes a Salesforce user request is raised as an **Incident**.

The process is:

Incident



Understand Request



Determine Whether It Is a Salesforce User Request



Request Additional Information



Provide Existing Excel Template



Requester Completes Template



Validate Required Information



Convert Incident → Service Request



Continue Salesforce User Request Process

Detailed process:

1. Receive the Incident.
2. Understand the user's requirement.
3. Identify whether the Incident is actually a Salesforce user request.
4. Request the additional required information.
5. Provide/use the existing Excel template.
6. Receive the completed information.
7. Validate whether all required information has been provided.
8. If sufficient:
 - Recommend/perform the Incident → Service Request conversion according to the existing process.
9. Continue with the Salesforce user-request workflow.

The AI assistant should help identify whether additional information is required and recommend the next appropriate action.

6. Primary Objective of the AI Application

The application should act as an:

AI Salesforce User Onboarding & Access Request Assistant

The AI assistant should help the support engineer with:

Ticket Understanding

- Understand the ticket/request
- Identify request type
- Extract relevant information

Information Validation

- Identify missing information
- Identify missing mirror-user information
- Identify missing business justification
- Identify incomplete request details

Approval Analysis

- Check Stage 1 approval
- Check Stage 2 approval
- Identify applicable Chatter approval
- Identify pending approvals

Business Rule Analysis

- Determine whether Chatter-specific conditions apply
- Determine whether mirror-user information is required
- Determine whether additional information is required

Next Action Recommendation

Examples:

- Request mirror user
- Request business justification
- Request additional details
- Request manager approval
- Wait for Stage 1 approval
- Wait for Stage 2 approval
- Proceed to next support step
- Recommend Incident → Service Request conversion

Communication Assistance

The AI may generate draft communication such as:

- Requester email/message
- Manager approval request

- Missing-information request
- Additional-detail request

The human support engineer must review and approve the communication before it is sent.

7. AI Does NOT Automatically Perform Production Actions

The initial architecture must follow a **Human-in-the-Loop** model.

The AI should recommend actions rather than independently making production changes.

For example:

Ticket



AI Analysis



AI Recommendation



Support Engineer Review



Human Approval



Human Performs Action

The AI should NOT independently:

- Create Salesforce users
- Assign permission sets
- Change Salesforce configuration
- Send production emails
- Change ticket status
- Convert production tickets
- Approve requests

unless explicitly introduced as a future capability and approved by the organization.

8. Input Sources

The application may use the following sources as knowledge/context.

A. Zoho Service Desk

Primary operational source.

Potential information includes:

- Onboarding tickets
- Salesforce tasks
- Salesforce New User Request tickets
- Incidents
- Service Requests
- Ticket conversations
- Ticket status
- Requester information
- Approval information
- Historical ticket information

The exact fields and API capabilities must be determined through the questionnaire.

B. Requirements / Business Documentation

Examples:

- Business requirements
- Functional requirements
- User stories
- Process documentation
- Salesforce onboarding procedures
- Access-management procedures

These may be embedded and stored in the retrieval layer where appropriate.

C. Existing Documentation

Examples:

- Confluence
- Wiki
- Technical documentation
- Salesforce support documentation

- Process documents
 - Knowledge articles
-

D. Existing Historical Tickets

Historical resolved tickets may be used to understand:

- Similar requests
- Previous resolutions
- Typical missing information
- Common support actions
- Historical communication patterns

Only approved and appropriate information should be used.

E. Existing Excel Templates

The current Excel template used for collecting additional Salesforce user-request information may be used as an input/reference source.

The architecture should determine how the Excel information will be extracted and validated.

F. Meeting Notes / Documentation

Where relevant:

- Meeting notes
- Meeting transcripts
- Process discussions

These may be converted into searchable knowledge.

9. Initial Knowledge Architecture

The initial concept is:

Approved Data Sources



Data Extraction



Cleaning / Normalization



Chunking



Metadata Extraction



Embedding



Vector Database

The initial proposed vector database is:

MongoDB

However, do not assume this is the final decision.

Use the questionnaire to determine whether MongoDB is appropriate.

10. Retrieval Architecture

The initial proposed retrieval architecture is:

Ticket / User Query



Query Preprocessing



Normalization



Abbreviation Expansion



Synonym Expansion



Vector Search



Keyword / BM25 Search



Hybrid Search



Reranking



Deduplication



Relevant Context



LLM



AI Analysis



Next Action Recommendation

Do not automatically include every retrieval technique.

Determine the appropriate architecture through the questionnaire.

11. Expected AI Output

The AI output should be structured.

Example:

Request Type:

Onboarding

Current State:

Business justification received

Mirror User:

Missing

Chatter User:

Yes

Chatter Status:

Pending

Stage 1 Approval:

Completed

Stage 2 Approval:

Completed

Missing Information:

Mirror user

Pending Action:

Request mirror-user information

Recommended Next Step:

Contact requester for mirror-user details

Confidence:

High

Supporting Evidence:

[Relevant ticket/document references]

The exact output structure should be finalized through the questionnaire.

12. Explainability Requirement

The AI should not simply say:

"Request mirror user."

It should explain **why**.

Example:

Reason:

The request is for a Chatter user and the mirror-user information required by the onboarding process is missing.

Recommended action:

Request the mirror-user information from the requester.

Where possible, the AI should provide references to the source information used for its recommendation.

13. RAG Requirement

The architecture should evaluate whether a **Retrieval-Augmented Generation (RAG)** approach is appropriate.

Potential knowledge sources:

Zoho Tickets

+

Process Documentation

+

Historical Tickets

+

Knowledge Articles

+

Excel Template

+

Technical Documentation

↓

Retrieval

↓

LLM

↓

AI Support Recommendation

The final architecture should explain:

- What should be embedded
- What should not be embedded
- Metadata strategy
- Chunking strategy

- Retrieval strategy
 - Context selection
 - Source citation
 - Data freshness
-

14. Proposed Technology Stack

Initial preferences:

Backend

Node.js + TypeScript

Frontend

React.js

Database / Vector Database

MongoDB

LLM

Groq and/or OpenAI

Embedding Model

OpenAI and/or Mistral

Deployment

AWS

Workflow / Agent Framework

To be determined.

Do not assume these technologies are final.

The questionnaire should validate the appropriate technology choices.

15. Architecture Quality Requirements

The final architecture should consider only requirements relevant to this application, including:

- Security
- Authentication
- Authorization
- Data privacy
- PII protection

- Prompt injection protection
- Hallucination control
- Human approval
- Auditability
- Observability
- Error handling
- Scalability
- Maintainability
- Cost
- Performance
- Retrieval accuracy
- AI response evaluation

Do not add unnecessary enterprise components simply for complexity.

16. Questionnaire Requirement

Before designing the architecture, ask me **exactly 20 architecture-discovery questions**.

Ask **ONE question at a time**.

Each question must contain:

Question X of 20

Question

A clear explanation of why the answer matters, when useful.

Options

- A.** Option A
- B.** Option B
- C.** Option C
- D.** Option D

Then wait for my answer.

Do not ask the next question until I answer the current question.

17. Question Scope

The 20 questions should be directly relevant to this application.

Questions may cover areas such as:

- Users of the application
- Zoho Service Desk integration
- Ticket types
- Ticket fields
- Business rules
- Approval handling
- Salesforce information
- Historical tickets
- Knowledge sources
- Excel template processing
- Data volume
- Data freshness
- RAG requirements
- Metadata
- Vector database
- Retrieval strategy
- LLM selection
- Human-in-the-loop
- Security
- PII
- Evaluation
- Deployment
- Monitoring
- Scalability

Do NOT ask questions about unrelated QA automation or test-case generation.

18. Unknown Answer Handling

If I do not understand a question, I may say:

Explain this.

You should explain the question in simple professional language and explain the implications of each option.

If I still do not understand, I may say:

You choose.

In that situation:

1. Recommend the most appropriate option.
 2. Explain why.
 3. Record the decision.
 4. Continue to the next question.
-

19. Skip Handling

If I believe a question is irrelevant, I may say:

Skip

In this case:

- Mark the question as **Skipped**.
- Do not force an answer.
- Do not invent an answer.
- Do not make assumptions to fill the gap.
- Continue to the next relevant question.

The final architecture must clearly distinguish between:

Answered / Recommended / Skipped / TBD

20. Strict No-Hallucination Rule

This is mandatory.

Do not invent:

- Huron internal architecture
- Zoho APIs
- Salesforce APIs
- Ticket fields
- Approval rules
- User roles
- Security policies
- Data volumes

- Existing integrations
- Infrastructure
- Business rules
- Internal Huron processes

If something is unknown, mark it as:

TBD — To Be Confirmed

Do not silently assume information.

21. Final Deliverable

After all 20 questions are completed, create a downloadable:

AI_Salesforce_User_Onboarding_Assistant_Architecture.md

The Markdown document should contain:

1. Executive Summary

2. Business Problem

3. Current-State Workflow

Document:

- Onboarding ticket
- Salesforce task
- Business justification
- Mirror user
- Chatter user
- Manager approval
- Salesforce New User Request
- Stage 1 approval
- Stage 2 approval
- Incident → Service Request

4. Proposed AI Solution

5. Goals

6. Non-Goals

7. Functional Requirements

8. Non-Functional Requirements

9. User Roles

10. End-to-End Architecture

Include a Mermaid architecture diagram.

11. Data Sources

12. Data Ingestion Pipeline

Explain:

Source

→ Extraction

→ Cleaning

→ Chunking

→ Metadata

→ Embedding

→ Storage

13. Retrieval Pipeline

Explain:

Query

→ Preprocessing

→ Retrieval

→ Reranking

→ Context

→ LLM

14. AI Decision / Recommendation Flow

Explain how the AI determines:

- Missing information
- Missing approvals
- Applicable conditions
- Next action
- Communication recommendation

15. RAG Architecture

16. LLM Architecture

17. Prompt Architecture

18. Human-in-the-Loop Architecture

19. Security Architecture

20. Data Privacy / PII

21. API Architecture

Include only APIs that are actually required.

22. Data Model

Include only relevant entities.

23. AWS Deployment Architecture

24. Monitoring & Observability

25. Error Handling & Failure Recovery

26. AI Evaluation

Include relevant measurements such as:

- Recommendation accuracy
- Retrieval relevance
- Missing-information detection accuracy
- Hallucination rate
- Human acceptance rate
- Response latency

Do not invent target values unless I provide them.

27. Cost Considerations

Provide conceptual cost considerations without inventing exact AWS costs.

28. Scalability

29. Risks & Mitigations

30. Future Enhancements

31. Final Architecture Decision Table

Architecture Area Final Decision Reason

Frontend

Backend

LLM

Embedding

Vector DB

Retrieval

Workflow

Deployment

Security

All final decisions must be based on my questionnaire answers.

22. Architecture Philosophy

The final solution should demonstrate:

Zoho Service Desk



Ticket Understanding



Business Context Retrieval



Business Rule Analysis



Missing Information / Approval Detection



AI Recommendation



Human Review



Requester / Manager Communication



Salesforce Support Action

The architecture should demonstrate how GenAI can **assist and improve an existing Salesforce support process**, rather than replacing the support engineer.

23. START THE QUESTIONNAIRE

Start immediately with:

Question 1 of 20

Ask only the first relevant architecture question.

Wait for my answer before asking Question 2.

Proposed Output

AI Salesforce User Onboarding & Access Request Assistant

Architecture design — TestLeaf ICEPOT assignment adapted to Huron workflow

****Status:**** Proposed design based on 20 questionnaire responses; not an approved Huron architecture.

****Scope:**** Zoho Service Desk Salesforce onboarding, Salesforce New User Request, and Salesforce user-request incidents. ****Not a test-case generator.****

1. Executive summary

A React-based AI workspace uses Node.js/TypeScript orchestration to read current Zoho and, where authorized, Salesforce data; retrieve approved process knowledge from MongoDB; evaluate structured business rules; and recommend or execute authorized workflow actions. An agent may select tools dynamically, but every action passes a policy gate. The organization must approve integrations, permissions, data processing, and deployment before production use.

2. Business problem

Support engineers manually inspect tickets, Salesforce tasks, business justification, mirror-user details, Chatter status, approvals, and Excel request templates. Missing information triggers requester/manager communications; some incidents must become service requests. The proposed assistant reduces repetitive interpretation and coordination while preserving auditable, policy-controlled decisions.

3. Current-state workflows

A. Onboarding ticket

Receive onboarding ticket → locate Salesforce task in main ticket → check mirror user and business justification → request missing details → after justification is received, check Chatter user and Pending status → if applicable, request manager approval in the same ticket → when all applicable conditions are met, proceed with Salesforce user creation.

B. Salesforce New User Request template

Receive request → verify Stage 1 and Stage 2 approvals → check mirror user and request missing value → if Chatter user with Pending status, request/verify applicable approval → if not Chatter, do not impose Chatter-specific approval → proceed only after all applicable conditions are met.

C. Incident raised for Salesforce user request

Identify user request → request additional details using existing Excel template → parse and validate completed template → when complete, convert incident to service request → continue applicable request workflow.

****Important:**** Exact Zoho field names, approval statuses, Chatter meaning, manager identity source, and Salesforce creation steps are TBD; no invented field mappings are implied.

4. Proposed solution, goals and non-goals

****Goals:**** classify request type; extract current state; detect missing fields and approvals; consult current rules and historical knowledge; resolve uncertainty with authorized live lookups; produce evidence-linked next actions and draft messages; execute permitted actions under configurable policy; maintain end-to-end audit.

****Non-goals:**** test-case generation; bypassing approvals; treating historical tickets as policy; allowing the LLM to grant its own permissions; unapproved access to production data.

5. Functional requirements

1. SSO-authenticated workspace for support engineers, QA engineers, leads and managers.

2. Near-real-time Zoho synchronization, with fresh reads immediately before consequential actions.
3. Ticket classification and structured extraction with provenance and unknown states.
4. RAG over approved process documents, wiki, technical material and permitted historical cases.
5. Structured policy evaluation for mirror user, justification, approvals, Chatter and conversion.
6. Dynamic agent tool selection subject to explicit allowlists and policy checks.
7. Excel attachment parsing, schema validation, missing-field feedback and conversion eligibility.
8. Draft requester/manager communications and, if authorized, execute them.
9. Salesforce user creation only through approved tools and authorization gates.
10. Explainable outcome with source links, rule version, action status and audit reference.
11. Escalation when evidence is insufficient, contradictory or tool execution fails.

6. Non-functional requirements

Role-based access; least privilege; encrypted transport/storage; PII minimization; audit retention; idempotent actions; retries with limits; source freshness; monitoring; versioned rules/prompts/models; tenant/project isolation where required. Actual SLA, throughput, retention, RTO/RPO, budget and compliance requirements are ****TBD****.

7. User roles

Support engineer: inspect, review, initiate approved workflow actions. QA engineer: evaluate outputs and test workflows with authorized data. Lead/manager: review, approve where designated, audit and configure authorized policies. Exact RBAC matrix and approval delegation are TBD.

8. End-to-end architecture

```mermaid

flowchart TD

U[React AI workspace] --> API[Node.js / TypeScript API]

Z[Zoho Service Desk] --> SYNC[Event or polling sync adapter]

SYNC --> OP[(Operational ticket state)]

API --> ORCH[Agent orchestrator]

ORCH --> TOOLS[Approved tool registry]

TOOLS --> Z

TOOLS --> SF[Salesforce - authorized live read/write]

TOOLS --> OP

ORCH --> RAG[Retrieval service]

DOC[Approved documents / wiki / historical cases] --> ING[Ingestion pipeline]

ING --> VDB[(MongoDB knowledge + vector indexes)]

RAG --> VDB

ORCH --> RULES[Versioned business rules]

ORCH --> LLM[Groq / OpenAI - selection TBD]

ORCH --> GATE[Policy and authorization gate]

GATE --> |Approved and permitted| TOOLS

GATE --> |Approval needed| U

ORCH --> AUDIT[(Append-only audit records)]

GATE --> AUDIT

TOOLS --> AUDIT

...

**\*\*Design correction to questionnaire choices:\*\*** “fully autonomous” describes the agent's ability to plan and invoke tools, **\*\*not\*\*** unrestricted authority. The selected configurable gates always apply. A model's dynamic source choice cannot override configured source ownership or verified live facts.

**## 9. Data sources and source-of-truth policy**

- Zoho: ticket, task, conversation and workflow/approval state where the system owns those records.
- Salesforce: live Salesforce user/access facts where authorized.
- Approved documentation: current process and policy definitions, versioned.
- Historical tickets: examples only, never overriding current rules.
- Excel: current request data, validated against a maintained schema.
- Other approved live tools: discover and authorize individually; no assumption that an integration exists.

For each field, configure `owner\_system`, freshness threshold and conflict policy. The agent may **\*\*select the relevant source dynamically\*\***, but a deterministic resolver checks ownership, timestamp and evidence. Unresolved conflicts block consequential writes and escalate. Exact hierarchy is TBD until field ownership is confirmed.

## ## 10. Ingestion pipeline

```
```mermaid
```

flowchart LR

S[Approved sources] --> C[Connector / extractor]

C --> P[Permission, PII and quality checks]

P --> N[Normalize and metadata]

N --> CH[Document-aware chunking]

CH --> E[Embedding model]

E --> M[(MongoDB vector index)]

N --> K[(Versioned source metadata)]

```
```
```

Store source ID, version, updated time, access labels, document type, process type, chunk ID and hash. Upsert changed versions; remove/revoke obsolete chunks; prevent unauthorized retrieval. Operational ticket states and approval values should remain structured and freshly queryable, **\*\*not be inferred solely from embeddings\*\***. Event-driven Zoho updates where supported; version/change-based

document reingestion. Webhook vs polling is TBD pending verified integration capability.

## ## 11. Retrieval pipeline

Ticket/query → normalization and safe abbreviation expansion → access-filtered metadata lookup → vector + keyword search (if supported/configured) → hybrid fusion → rerank → deduplicate → filter obsolete/revoked content → compact context with citations → LLM. Search implementation, weights, chunk sizes, embedding model, reranker and token budget are TBD; benchmark on approved examples rather than assuming every component is necessary.

## ## 12. Agent decision and action pipeline

```mermaid

flowchart TD

A[Ticket event / user request] --> B[Fetch fresh Zoho state]

B --> C[Classify and extract structured facts]

C --> D[Select approved live lookups and RAG]

D --> E[Resolve provenance / conflicts]

E --> F[Evaluate versioned rules]

F --> G{Evidence sufficient?}

G -->|No| H[Ask for clarification / escalate]

G -->|Yes| I[Plan next action and draft message]

I --> J[Policy gate: identity, role, action, ticket, risk]

J -->|Human approval required| K[Await authorized approval]

K --> L[Re-fetch state and revalidate]

J -->|Pre-authorized| L

L --> M{Still eligible?}

M -->|No| H

M -->|Yes| N[Execute allowlisted tool]

N --> O[Verify outcome / audit / update UI]

...

Use deterministic validation for critical conditions; RAG supplies explanations, not binding approval status. Agent plans are proposals until validated. For any uncertain status, use `unknown` rather than assuming approval.

13. Workflow decision matrix (illustrative, subject to field mapping)

| Workflow | Observed condition | Proposed next action |

| --- | --- | --- |

| Onboarding | Salesforce task not found | Flag for engineer investigation; do not assume creation eligibility |

| Onboarding | Mirror user missing | Request mirror-user information |

| Onboarding | Business justification missing | Request justification |

| Onboarding | Chatter and Pending after justification | Request manager approval in same ticket |

| New user template | Stage 1 or Stage 2 incomplete | Wait/request applicable approval; no creation |

| New user template | Mirror user missing | Request mirror user |

| New user template | Chatter and Pending | Request/verify applicable approval |

| New user template | Not Chatter | Skip Chatter-specific approval |

| Incident | Excel details incomplete | Request missing template fields |

| Incident | Excel details complete | Check policy and convert to service request |

| Any | Conflicting/unknown critical facts | Fetch authoritative source, then escalate if unresolved |

Multiple missing conditions can coexist; the configured workflow policy determines communication grouping and action order. The current narrative does not establish every possible status transition.

14. Structured output contract (illustrative)

```json

```

{
 "ticket_id": "example-only",
 "request_type": "onboarding|new_user_template|incident|unknown",
 "facts":
 [{"name":"mirror_user","value":null,"status":"missing","source":"ticket"}],
 "approvals": [{"stage":"stage_1","status":"unknown","source":null}],
 "missing_information": ["mirror_user"],
 "recommended_actions": [{"type":"request_information","reason":"Mirror user
absent","requires_approval":true}],
 "draft_message": null,
 "evidence": [],
 "rule_version": "TBD",
 "requires_escalation": false
}
...

```

Actual schema, mandatory fields and enumerations require validation against real sanitized ticket/template samples.

## **## 15. RAG, LLM and prompt architecture**

Use separate instructions for classification/extraction, evidence-based reasoning and communication drafting. Require structured JSON/schema validation, explicit unknowns, citations to retrieved source IDs, and rejection of instructions embedded in ticket text or documents. Keep secrets and policy enforcement outside prompts. OpenAI/Groq LLM and OpenAI/Mistral embeddings remain candidates, not verified organizational approvals. Compare quality, cost, data handling, latency and contractual constraints before selection.

## **## 16. Human-in-the-loop and autonomous execution**

Questionnaire selects autonomous agent plus configurable approval policies. Implement policy-as-code: `deny`, `allow`, or `require\_approval`. Require authenticated approver identity, ticket/action binding, expiry, approval evidence,

fresh-state recheck and idempotency key. Production user creation, permission changes, ticket conversion and external messages must never be executed merely because an LLM requested them. Initial safe rollout should use read-only/shadow mode before enabling writes; actual action permissions are TBD.

## ## 17. Security and privacy

SSO/OIDC subject to enterprise approval; service credentials in secrets manager; separate read/write scopes; redact or minimize personal data before embeddings/LLM calls; encrypt in transit/at rest; ACL-filter retrieval; guard against prompt injection and cross-ticket leakage; use sanitized fixtures for coursework. Determine vendor data residency, retention, logging and DPA requirements with organizational security owners before connecting production systems.

## ## 18. API architecture (proposed internal contracts)

- `GET /tickets/{id}/analysis`: current structured analysis.
- `POST /tickets/{id}/analyze`: initiate fresh analysis.
- `GET /tickets/{id}/recommendations`: recommendations and evidence.
- `POST /actions/{id}/approve`: authorized approval.
- `POST /actions/{id}/execute`: policy-gated execution with idempotency.
- `POST /ingestion/events`: authenticated source update callback if supported.

These are **\*\*application-owned proposed endpoints\*\***, not claims about Zoho/Salesforce API names. External connector endpoints and scopes are TBD.

## ## 19. Data model

`TicketSnapshot(ticketId, type, status, sourceVersion, fetchedAt)`;  
`ExtractedFact(name, value, evidence, confidence, observedAt)`; `Approval(stage, state, authoritativeSource, updatedAt)`; `KnowledgeChunk(sourceId, version, ACL, embedding, hash)`; `Rule(ruleId, version, conditions, action)`; `AgentRun(runId, ticketId, modelVersion, promptVersion)`; `ProposedAction(actionId, risk, status, idempotencyKey)`; `ApprovalDecision(actor, policyVersion, time)`;  
`AuditEvent(correlationId, eventType, immutablePayloadRef)`. Exact physical schema and retention are TBD.

## **## 20. AWS deployment (proposed, not current Huron infrastructure)**

**```mermaid**

**flowchart TD**

**B[Browser] --> W[Approved web ingress / WAF]**

**W --> FE[React static hosting]**

**W --> BE[Node.js TypeScript API / workers]**

**BE --> Q[Queue / event bus]**

**Q --> WRK[Sync and ingestion workers]**

**BE --> DB[(MongoDB deployment - selection TBD)]**

**WRK --> DB**

**BE --> SEC[Secrets / IAM / KMS]**

**BE --> OBS[Logs, metrics, traces, audit sink]**

**BE --> EXT[Approved Zoho, Salesforce, LLM connectors]**

**```**

**AWS service selection (e.g., ECS/Lambda, S3, SQS/EventBridge, CloudWatch) depends on network/security review, expected load and deployment approval. MongoDB hosting and vector-search feature availability must be verified; do not assume a particular MongoDB product or search capability.**

## **## 21. Observability and audit**

**Capture correlation ID, ticket ID (access-controlled), connector request outcome, retrieval source/version, rule evaluation, model/prompt version, action plan, policy result, approver, execution response and verification. Maintain append-only/tamper-evident records and separate restricted audit access. Avoid raw PII in telemetry. “Immutable” requires an actual approved storage/retention configuration; design intent alone does not guarantee immutability.**

## **## 22. Failure handling**

**Zoho sync failure → mark stale and re-fetch before action. Salesforce timeout → query current state before retry to avoid duplicate user creation. LLM/schema failure → retry within limit, then escalate. Missing source or rule conflict → no**

write. Duplicate events → deduplicate by event/source version. Expired approval → reapprove. Failed ticket conversion or email → record outcome and reconcile before retry.

## **## 23. Evaluation and release gates**

Use sanitized, expert-labeled cases across all three workflows. Measure request classification, missing-field detection precision/recall, approval-rule correctness, next-action agreement, retrieval relevance, citation support, unsupported claims, human acceptance, latency, tool-action correctness and incident rate. Include negative/conflicting cases and regression tests for rule changes. Pilot in shadow mode, then gated actions, then expand only with approval. Numerical thresholds are TBD.

## **## 24. Cost and scalability**

Cost drivers: connector polling/events, embeddings/reindexing, LLM input/output tokens, vector storage/search, reranking, AWS compute/queue/observability and audit retention. Reduce cost with change-based ingestion, caching of non-sensitive stable knowledge, retrieval limits and model routing. Scale ingestion and agent workers independently; enforce per-system API rate limits. No cost or capacity figures are assumed.

## **## 25. Risks and mitigations**

- Stale approval data → live recheck before writes.
- LLM hallucination → structured extraction, source evidence and deterministic rule validation.
- Overprivileged agent → allowlisted tools and external policy gate.
- Historical policy drift → versioned current rules take precedence.
- PII exposure → access filtering, minimization and approved vendors.
- Duplicate user creation → idempotency and post-action verification.
- Source conflicts → authoritative field ownership and escalation.

## **## 26. Phased delivery**

1. **\*\*MVP/shadow mode:\*\*** React workspace, Zoho read integration, approved documents, RAG, structured rules, drafts, audit.
2. **\*\*Controlled workflow:\*\*** Excel validation, sync, evaluation suite, authorized ticket updates and communications behind gates.
3. **\*\*Extended actions:\*\*** approved Salesforce read/write and incident conversion with policy checks, reconciliation and monitoring.

All phases require internal approval and verified integration capability.

## **## 27. Architecture decision record — 20 questionnaire answers**

| <b>  Q  </b>   | <b>Selected answer  </b> | <b>Design implication  </b>                                              |
|----------------|--------------------------|--------------------------------------------------------------------------|
| <b>  ---  </b> | <b>  ---  </b>           | <b>  ---  </b>                                                           |
| <b>  1  </b>   | <b>D  </b>               | <b>Support, QA, leads/managers  </b>                                     |
| <b>  2  </b>   | <b>B  </b>               | <b>Zoho API/integration  </b>                                            |
| <b>  3  </b>   | <b>D  </b>               | <b>Analysis, next action, communication drafts  </b>                     |
| <b>  4  </b>   | <b>D  </b>               | <b>Broad approved knowledge sources  </b>                                |
| <b>  5  </b>   | <b>D  </b>               | <b>Rules, historical cases, resolutions, documentation  </b>             |
| <b>  6  </b>   | <b>D  </b>               | <b>Near-real-time Zoho synchronization  </b>                             |
| <b>  7  </b>   | <b>D  </b>               | <b>Agent selects relevant tools/rules  </b>                              |
| <b>  8  </b>   | <b>D  </b>               | <b>Autonomous execution, constrained by later policy choices  </b>       |
| <b>  9  </b>   | <b>D  </b>               | <b>Cross-system actions with configurable gates  </b>                    |
| <b>  10  </b>  | <b>D  </b>               | <b>Configurable approval/authorization policies  </b>                    |
| <b>  11  </b>  | <b>D  </b>               | <b>Dynamic approved live-source and knowledge retrieval  </b>            |
| <b>  12  </b>  | <b>D  </b>               | <b>Dynamic source choice; conflict policy must still be configured  </b> |
| <b>  13  </b>  | <b>D  </b>               | <b>Structured rules + RAG + agent  </b>                                  |
| <b>  14  </b>  | <b>D  </b>               | <b>Excel validation and incident-conversion support  </b>                |
| <b>  15  </b>  | <b>D  </b>               | <b>Historical learning; current facts/rules primary  </b>                |
| <b>  16  </b>  | <b>D  </b>               | <b>Investigate uncertainty, then escalate  </b>                          |
| <b>  17  </b>  | <b>D  </b>               | <b>Event-driven operational sync + version-based documents  </b>         |
| <b>  18  </b>  | <b>D  </b>               | <b>Full versioned audit trail  </b>                                      |

| 19 | D | Unified React AI workspace |

| 20 | D | Comprehensive automated/human evaluation |

## **## 28. Final technology decisions**

| Area | Decision | Status / rationale |

| --- | --- | --- |

| Frontend | React | User-proposed; unified workspace selected |

| Backend | Node.js + TypeScript | User-proposed |

| LLM | Groq or OpenAI | TBD after approval and evaluation |

| Embeddings | OpenAI or Mistral | TBD after evaluation |

| Vector store | MongoDB | Proposed; verify deployment/search capabilities |

| Retrieval | Access-filtered hybrid where supported | Benchmark before finalizing |

| Workflow | Agent orchestrator + structured rules + external policy gate | Reflects answers 7–13 |

| Integrations | Zoho primary; Salesforce and others if approved | Endpoint/field mapping TBD |

| Deployment | AWS | User-proposed; exact services TBD |

| Security | RBAC, least privilege, configurable gates, audit | Detailed enterprise policies TBD |

## **## 29. Open decisions / information required before implementation**

Actual sanitized ticket and Excel samples; Zoho product/version/API access and webhook availability; Salesforce read/write scope; Stage 1/2 and Chatter approval semantics; mirror-user validation criteria; authoritative ownership for each field; manager identity lookup; exact creation/incident-conversion procedure; permitted knowledge sources; data classification and LLM vendor approval; IAM/SSO; retention; throughput, SLA, budget and rollout owners.

**\*\*Architecture principle:\*\* live facts + current versioned business rules determine eligibility; RAG provides relevant evidence; the agent proposes actions; independent authorization controls execution.**

